

SearchVision

Human-in-the-Loop Object Detection Dataset Generation
from Web Images

White Paper — v0.3

Brandon Shen

July 2026

Licensed under AGPL-3.0 • Open-Source •
<https://github.com/Brandon-Shen/SearchVision>

Abstract

SearchVision is a web-based platform that enables users to create custom object detection models without writing code. By integrating web search (with intelligent fallback strategies), automated web scraping, intelligent annotation workflows, and YOLOv8 model training into a unified workflow, SearchVision automates the traditionally labor-intensive process of dataset collection, annotation, and model training. Unlike existing solutions that focus on individual pipeline stages, SearchVision integrates all five stages—search, selection, annotation, scraping, and training—into a single interface. The system features a hybrid image selection algorithm combining search rank, caption relevance, and visual diversity; validated image downloads; a held-out validation split; and conservative dataset augmentation using a pre-trained YOLOv8 model. Google Custom Search is optional because an original-resolution Bing Images fallback is provided. This whitepaper presents the system’s architecture, feature set, technology stack, and practical use cases, while also acknowledging current limitations.

Contents

1	Introduction	1
1.1	Installation and Quick Start	1
2	System Architecture	2
2.1	Component Breakdown	2
2.1.1	Frontend Layer (Presentation)	3
2.1.2	Backend Layer (Application Logic)	3
2.1.3	Data Layer (Storage)	4
2.1.4	Annotation JSON Schema	4
2.1.5	YOLO Format Conversion	4
2.1.6	REST API Endpoints	5
2.2	Data Flow Sequence	5
2.3	System Architecture Diagram	7
3	Feature Breakdown	7
3.1	Intelligent Image Search	8
3.1.1	Algorithmic Details: Balanced Selection	8
3.2	Automated Web Scraping	10
3.2.1	Scraping Methodology	11
3.3	Dual-Mode Annotation	12
3.3.1	Manual Annotation: Canvas Interface	12

3.3.2	Auto-Annotation: Class-Filtered YOLOv8m Pipeline	13
3.4	One-Click Model Training	14
3.4.1	Training Configuration and Hyperparameters	14
3.4.2	Validation Method and July 2026 Smoke Benchmark	15
3.4.3	Data Augmentation and Small-Dataset Handling	16
4	Technology Stack	18
5	Related Work	18
5.1	Annotation Platforms	19
5.2	AutoML and Cloud-Based Training	19
5.3	Dataset Search and Analysis Tools	19
5.4	Web Scraping and Image Collection	20
6	Use Cases	20
6.1	Rapid Prototyping	20
6.2	Startup Bootstrapping	21
6.3	Educational Purposes	21
6.4	Research Iteration	22
7	Limitations and Known Issues	23
8	Conclusion	25

1. Introduction

The proliferation of deep learning-based computer vision has enabled remarkable advances in object detection, with applications spanning autonomous vehicles, medical imaging, retail analytics, and industrial quality control. State-of-the-art models such as YOLO [Redmon and Farhadi, 2018], Faster R-CNN [Ren et al., 2015], and their successors achieve strong performance on standard benchmarks. However, the deployment of these models to real-world problems remains challenging, particularly for practitioners outside of well-resourced research laboratories.

The primary bottleneck is not the model architecture itself, but rather the dataset required for training. Creating a custom object detection dataset involves four distinct challenges: (1) collecting relevant images, (2) annotating those images with bounding boxes, (3) ensuring dataset diversity and quality, and (4) configuring and executing the training pipeline. Each step traditionally requires specialized tools, domain expertise, and significant manual effort.

Several commercial and open-source solutions have emerged to address individual components of this workflow. Annotation platforms such as CVAT [CVAT, 2024], Label Studio [Label Studio, 2024], and Roboflow [Roboflow, 2024] provide sophisticated interfaces for bounding box creation. AutoML services from major cloud providers automate model training but at substantial cost. SearchVision contributes a compact, open-source integration of these stages rather than claiming that comparable integrated platforms do not exist.

This whitepaper presents SearchVision, a system that automates the complete object detection dataset creation and model training workflow. The system accepts a text query from the user, retrieves relevant images via web search, applies diversity-aware selection algorithms to ensure dataset variety, expands the dataset through automated web scraping, supports both manual and auto-annotation workflows, and trains a custom YOLOv8 model [Ultralytics, 2024] with a single click. By abstracting implementation details, SearchVision enables practitioners to focus on their domain problems rather than tooling configuration.

1.1 Installation and Quick Start

SearchVision is designed for local deployment on a machine with Python 3.12 and (optionally) GPU acceleration.

1. Clone the Repository.

```
git clone https://github.com/Brandon-Shen/SearchVision.git
cd SearchVision
```

2. Install Dependencies.

```
pip install -r requirements.txt
```

The requirements file includes FastAPI, Ultralytics, PyTorch, BeautifulSoup4, and other necessary packages.

3. Configure API Credentials (Optional).

Create a `.env` file in the project root to enable Google Custom Search (optional):

```
GOOGLE_API_KEY=your_google_api_key  
SEARCH_ENGINE_ID=your_custom_search_engine_id
```

The Google API key is obtained from Google Cloud Console; the Custom Search Engine ID from programmablesearchengine.google.com. If Google API is unavailable or disabled, the system automatically falls back to Bing Images web scraping, which does not require API keys and provides robust, rate-limited access to image results.

4. Launch the Application.

```
python -m uvicorn src.main:app --reload
```

The application will be available at `http://localhost:8000`.

5. Workflow Execution.

- a. Navigate to the web interface and enter a search query.
- b. Select relevant images from the diversity-filtered results.
- c. Draw at least one bounding box on each of at least five selected images.
- d. Click “Train Model” to initiate training.
- e. Download the trained model weights from the results page.

The complete workflow can be executed in a single session on consumer hardware with GPU support. Training time depends on available compute resources and batch size auto-adjustment.

2. System Architecture

2.1 Component Breakdown

SearchVision follows a modular architecture with three primary layers.

2.1.1 Frontend Layer (Presentation)

The frontend consists of six HTML templates rendered via Jinja2, each corresponding to a distinct workflow stage:

Table 1: Frontend Components

Component	File	Purpose
Base Template	<code>base.html</code>	Shared layout, navigation, and CSS
Search	<code>search.html</code>	Query input and image search interface
Selection	<code>select.html</code>	Image selection grid with diversity visualization
Annotation	<code>annotate.html</code>	Canvas-based bounding box drawing tool
Training	<code>training.html</code>	Real-time training progress display
Results	<code>results.html</code>	Trained model output and download

2.1.2 Backend Layer (Application Logic)

FastAPI serves as the web framework, providing RESTful endpoints and async request handling. The backend comprises the following modules:

Table 2: Backend Modules

Module	File	Responsibility
Search Integration	<code>search_images.py</code>	Google Custom Search API wrapper + Bing Images fallback
Image Download	<code>download_images.py</code>	Validated HTTP image download, RGB conversion, and local storage
Balanced Selection	<code>select_balanced_images.py</code>	Hybrid relevance + dissimilarity selection with ResNet50
Web Scraping	<code>scrape_similar.py</code>	Query- and selected-domain-based dataset expansion (up to 30 images)
Auto-Annotation	<code>auto_annotate_images.py</code>	Pre-trained YOLOv8 inference for bounding boxes
Model Training	<code>train_model.py</code>	YOLOv8 training orchestration and monitoring
Dataset Config	<code>create_data_yaml.py</code>	YOLO-format dataset YAML generation
Annotation Utils	<code>annotation_converter.py</code>	JSON-to-YOLO format conversion with robust error handling

2.1.3 Data Layer (Storage)

- **Local File System.** Images and YOLO labels are stored under `dataset/train/` and `dataset/val/` after a deterministic 80/20 split.
- **Configuration Files.** `data.yaml` for YOLO dataset configuration; `.env` for API credentials.

2.1.4 Annotation JSON Schema

Manual annotations are stored in JSON format before conversion to YOLO format. The schema records rectangles in 300-by-300 canvas coordinates together with the displayed image dimensions and letterbox offsets required to map them back to source-image coordinates:

Table 3: Annotation JSON Schema

Field	Type	Req.	Description
<code>rects</code>	array	Yes	Canvas rectangles with $x, y, width, height$
<code>canvasWidth</code>	number	Yes	Annotation canvas width
<code>canvasHeight</code>	number	Yes	Annotation canvas height
<code>imageElement</code>	object	Yes	Display size and x/y letterbox offsets

Example Annotation JSON:

```
{
  "rects": [{ "x": 75, "y": 60, "width": 90, "height": 140 }],
  "canvasWidth": 300,
  "canvasHeight": 300,
  "imageElement": {
    "displayWidth": 300, "displayHeight": 200,
    "offsetX": 0, "offsetY": 50
  }
}
```

2.1.5 YOLO Format Conversion

The annotation converter transforms pixel-coordinate bounding boxes to YOLO normalized format. For each bounding box:

1. Compute center coordinates:

$$x_{\text{center}} = \frac{x_{\min} + x_{\max}}{2 \times W}, \quad y_{\text{center}} = \frac{y_{\min} + y_{\max}}{2 \times H} \quad (1)$$

2. Compute normalized dimensions:

$$w_{\text{norm}} = \frac{x_{\text{max}} - x_{\text{min}}}{W}, \quad h_{\text{norm}} = \frac{y_{\text{max}} - y_{\text{min}}}{H} \quad (2)$$

The YOLO label file contains one line per bounding box in the form `<class_id> <x_center> <y_center> <width> <height>`. For the example above (`class_id = 0`):

```
0 0.162760 0.264 0.070312 0.306
0 0.487240 0.333 0.046875 0.296
```

2.1.6 REST API Endpoints

The FastAPI backend exposes the following endpoints for frontend communication:

Table 4: REST API Endpoints

Method	Endpoint	Description
GET	<code>/</code>	Home/search page (clears dataset)
POST	<code>/search</code>	Execute search with pagination, return diverse results
POST	<code>/select</code>	Handle image selection, download to local storage
GET	<code>/training-status</code>	Poll training progress (JSON)
POST	<code>/start-training</code>	Initiate training with annotations (async)
POST	<code>/save_annotations</code>	Save annotations and trigger training
GET	<code>/results</code>	Display trained model output and download
GET	<code>/download-model</code>	Download trained model weights (.pt file)
GET	<code>/error</code>	Error page with message query param

All endpoints return HTML responses (Jinja2 templates), except `/training-status` which returns JSON. Long-running operations (scrape, download, train) update a global status object that is polled by the frontend for real-time progress updates.

2.2 Data Flow Sequence

SearchVision Workflow Overview

Search → **Select** → **Annotate** → **Scrape & Train** → **Results**

1. Search Phase

- User enters a text query (e.g., “*traffic lights*”).
- Backend attempts Google Custom Search API; if unavailable, falls back to Bing Images.
- Up to 30 results are fetched; a 15-result page is considered at a time.
- Hybrid scoring applies 60% search rank, 25% caption relevance, and 15% visual dissimilarity.

2. Selection Phase

- User reviews 9 most diverse images presented in 3×3 grid.
- User selects a minimum of 5 images; “Search Again” enables pagination.
- Selected images downloaded to local storage.
- User advances to annotation phase.

3. Annotation Phase

- **Manual.** User draws bounding boxes on the canvas interface.
- Human boxes are retained as ground truth and are never overwritten.
- Annotations converted to YOLO format (normalized *center_x*, *center_y*, *width*, *height*).

4. Training Pipeline (Async)

- a. Search for up to 30 additional images using query modifiers and domains represented in the user’s selections.
- b. Download scraped images to local storage.
- c. Auto-annotate only scraped images for which the requested class maps to a known COCO class and a target detection is found. Uncertain images are excluded.
- d. Generate a deterministic 80/20 train/validation split and `data.yaml`.
- e. Fine-tune YOLOv8m for up to 75 epochs with automatic batch sizing and early stopping (patience 10). `YOLO_MODEL` can select another backbone.
- f. Export the exact run’s `weights/best.pt` artifact.

5. Results Phase

- Display the class, backbone, dataset counts, and configured epoch ceiling.
- Provide the trained PyTorch model file through a path-confined endpoint.
- “Train Another Model” button to restart workflow.

2.3 System Architecture Diagram

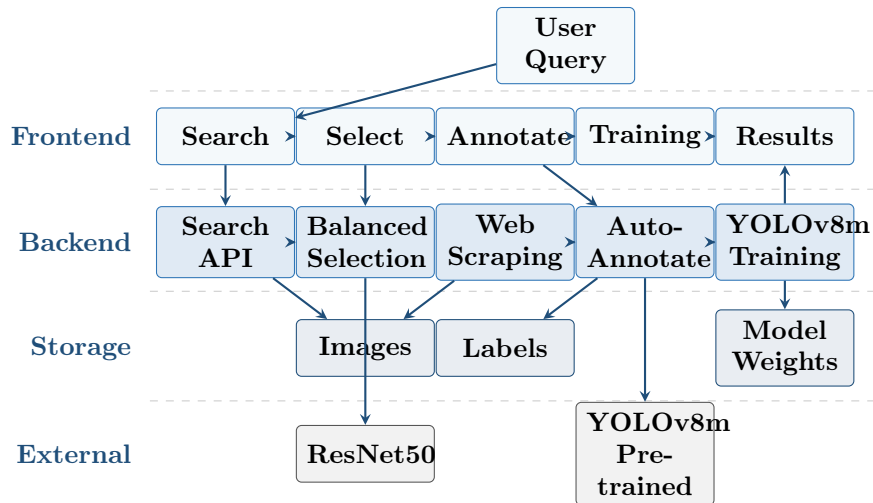


Figure 1: SearchVision system architecture. The workflow progresses left-to-right through five frontend pages (search, selection, annotation, training, results). The backend orchestrates image retrieval via Google Custom Search or Bing, diversity-aware selection using ResNet50 feature extraction, discovery of up to 30 additional images, conservative auto-annotation with pre-trained YOLOv8m, and final model training. The storage layer persists images, labels, and model weights; external dependencies provide pre-trained ResNet50 and YOLOv8m models.

Figure 1 illustrates the complete SearchVision workflow and system components. Data flows from the user query through the search and selection phases, manual annotation on the canvas, automatic augmentation via scraping and auto-annotation, model training, and finally model download. The architecture is modular, with clear separation between frontend (user-facing pages), backend (processing logic), storage (data persistence), and external dependencies (APIs and pre-trained models).

3. Feature Breakdown

SearchVision implements four major features that collectively enable end-to-end custom object detection model creation.

3.1 Intelligent Image Search

Description. Users enter a text query and SearchVision retrieves relevant images using Google Custom Search API with automatic fallback to Bing Images. The system filters up to 30 results and scores each displayed page using search position, caption relevance, and visual diversity.

Technical Implementation.

- Integration with Google Custom Search JSON API (primary) + Bing Images scraping (fallback).
- ResNet50 [He et al., 2016] pretrained model extracts 2048-dimensional feature vectors from the penultimate layer (avgpool).
- Scikit-learn cosine distance calculates pairwise visual dissimilarity.
- Hybrid scoring formula: $\text{Score}_i = 0.60R_i + 0.25C_i + 0.15D_i$.
- A one-pass top- k ranking selects up to 9 balanced images.

3.1.1 Algorithmic Details: Balanced Selection

The image selection problem is formalized as a **Hybrid Relevance-Diversity Optimization** problem, where up to nine images are selected from a page of candidates by balancing search rank, lexical caption relevance, and visual dissimilarity.

Feature Extraction. Each image i is passed through a pretrained ResNet50 model [He et al., 2016] to extract a feature vector $\mathbf{f}_i \in \mathbb{R}^{2048}$ from the average pooling layer, ensuring generic visual features suitable for cross-domain similarity measurement.

Hybrid Scoring. For each image i , we compute a composite score:

$$\text{Score}_i = 0.60R_i + 0.25C_i + 0.15D_i \tag{3}$$

where:

- R_i is normalized search-result position.
- C_i is normalized query-word overlap and position-weighted caption relevance.
- D_i is normalized summed cosine distance to the candidate pool.

Cosine Distance. Given two feature vectors $\mathbf{A}$ and $\mathbf{B}$:

$$\text{similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{j=1}^{2048} A_j B_j}{\sqrt{\sum_{j=1}^{2048} A_j^2} \cdot \sqrt{\sum_{j=1}^{2048} B_j^2}} \quad (4)$$

Cosine distance is defined as $d(\mathbf{A}, \mathbf{B}) = 1 - \text{similarity}(\mathbf{A}, \mathbf{B})$. Cosine distance is preferred over Euclidean distance because it measures angular similarity in the feature space, which is more robust to illumination variations and scale differences.

One-Pass Top-K Selection.

1. For each candidate image i in the pool of 30 search results:

$$\text{Score}_i = 0.60R_i + 0.25C_i + 0.15D_i \quad (5)$$

2. Compute summed dissimilarity $D_i = \sum_j d(\mathbf{f}_i, \mathbf{f}_j)$.
3. Sort by composite score and select top $k = 9$ images.

This one-pass ranking approach is computationally efficient but does not provide iterative diversity guarantees. The algorithm selects images with high overall scores (relevant + dissimilar) rather than greedily maximizing minimum pairwise distance between selected images.

Why ResNet50? ResNet50 [He et al., 2016] was chosen over lighter alternatives (e.g., MobileNet, EfficientNet) because:

1. **Feature Quality.** The 2048-dimensional features from ResNet50’s penultimate layer capture rich semantic information suitable for cross-domain similarity.
2. **Pre-trained Weights.** ImageNet pretraining provides generic visual priors.
3. **Inference Speed.** Despite being larger than mobile-optimized models, ResNet50 inference on a single image takes < 50 ms on modern GPUs, making it practical for batch processing of search results.

User Experience. Nine images are presented in a 3×3 grid. Users must select at least five images to proceed to annotation. Selected images are indicated with checkmark overlays.

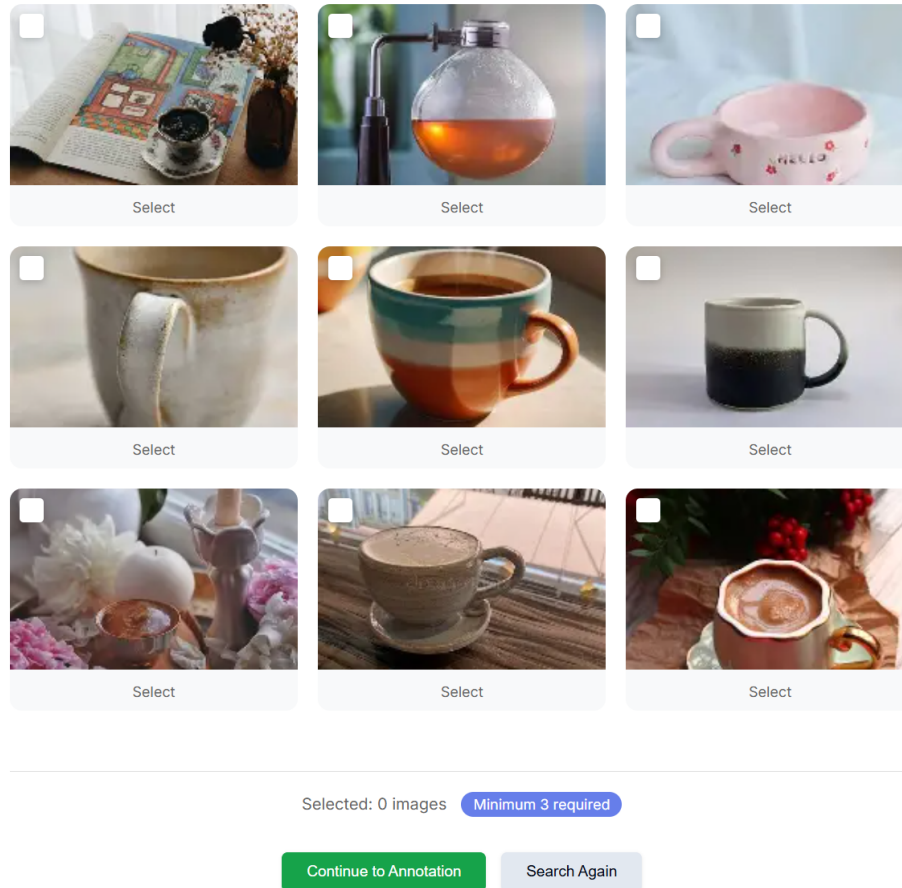


Figure 2: Example 3×3 diversity-selection grid output. The system uses ResNet50 feature extraction and cosine distance to select the nine most visually diverse images from search results, ensuring dataset variety.

Figure 2 illustrates the diversity selection interface, showing how SearchVision presents visually distinct images to maximize dataset coverage.

3.2 Automated Web Scraping

Description. After user selection and annotation, SearchVision automatically scrapes up to 30 additional candidates to expand the dataset. Expansion combines descriptive query variants with up to three source domains represented in the user’s selections. Candidates enter training only when a class-filtered pre-trained detector produces a target annotation.

Technical Implementation.

- Google Custom Search when configured, with a Bing Images fallback.

- BeautifulSoup extracts Bing’s original-resolution `url` metadata; a thumbnail parser is retained only as a compatibility fallback.
- Query variants include quality, isolation, product-photo, detail, and close-up modifiers plus selected source domains.
- Pillow decodes every HTTP 200 response and normalizes valid images to RGB JPEG.

3.2.1 Scraping Methodology

Primary Source: Bing Images. The scraper targets Bing Images as the primary source because:

1. **No API Key Required.** Unlike Google Custom Search, Bing Images can be scraped without API credentials.
2. **Reliable Fallback.** When Google Custom Search API is unavailable or rate-limited, Bing provides consistent access to diverse image results.
3. **Original Images.** The parser prefers publisher-hosted original URLs over low-resolution Bing thumbnails.
4. **Query Flexibility.** Bing search accepts various query formats and modifiers.

Query Expansion Strategy. To maximize coverage and find visually diverse training images, the scraper generates multiple query variations:

1. **Simple Query.** The original user query (e.g., “car”).
2. **Contextual Modifiers.** Quality and viewpoint-oriented phrases such as “clear photo,” “isolated,” and “close-up.”
3. **Selection Domains.** Up to three `site:` queries derived from the images explicitly chosen by the user.
4. **Fallback Queries.** Progressively simpler variants if complex queries return limited results.

Image Download and Storage. Downloads are validated before storage:

1. **HTTP Status Check.** Verify successful HTTP 200 response.
2. **Decode Check.** Pillow must parse the response as an image.
3. **Normalization.** Convert to RGB and save as a high-quality JPEG with collision-resistant selected/scraped prefixes.

The downloader does not yet impose a minimum resolution or byte-size ceiling; those remain quality and resource-control limitations.

Rate Limiting and Robustness. To avoid IP blocks and service interruptions:

- Search fallback retries up to three times with one-second delays.
- Request timeout set to 10 seconds.
- Graceful failure: If scraping fails, training continues with manual annotations only (no error thrown).

The expansion stage targets up to 30 additional images per training run. Duration depends on network conditions, source availability, and how many responses decode successfully.

User Experience. Fully automated—users see a progress indicator displaying current image count while images are scraped and downloaded.

3.3 Dual-Mode Annotation

SearchVision supports manual and automatic annotation approaches.

3.3.1 Manual Annotation: Canvas Interface

The annotation canvas is implemented with the HTML5 Canvas API:

1. **Image Rendering.** Images are scaled into a 300-by-300 canvas while preserving aspect ratio and recording letterbox offsets.
2. **Bounding Box Drawing.** `mousedown` records (x_1, y_1) ; `mousemove` draws a rubber-band rectangle; `mouseup` finalizes the box.
3. **Box Manipulation.** The current interface supports creating multiple boxes and clearing all boxes for an image before redrawing.
4. **Coordinate Transformation.** Screen coordinates are mapped to image space:

$$x_{\text{image}} = x_{\text{screen}} \times \frac{\text{image_width}}{\text{canvas_width}} \quad (6)$$

Boxes smaller than 10×10 canvas pixels are discarded. The application currently trains one class, so every accepted rectangle is assigned class ID 0.

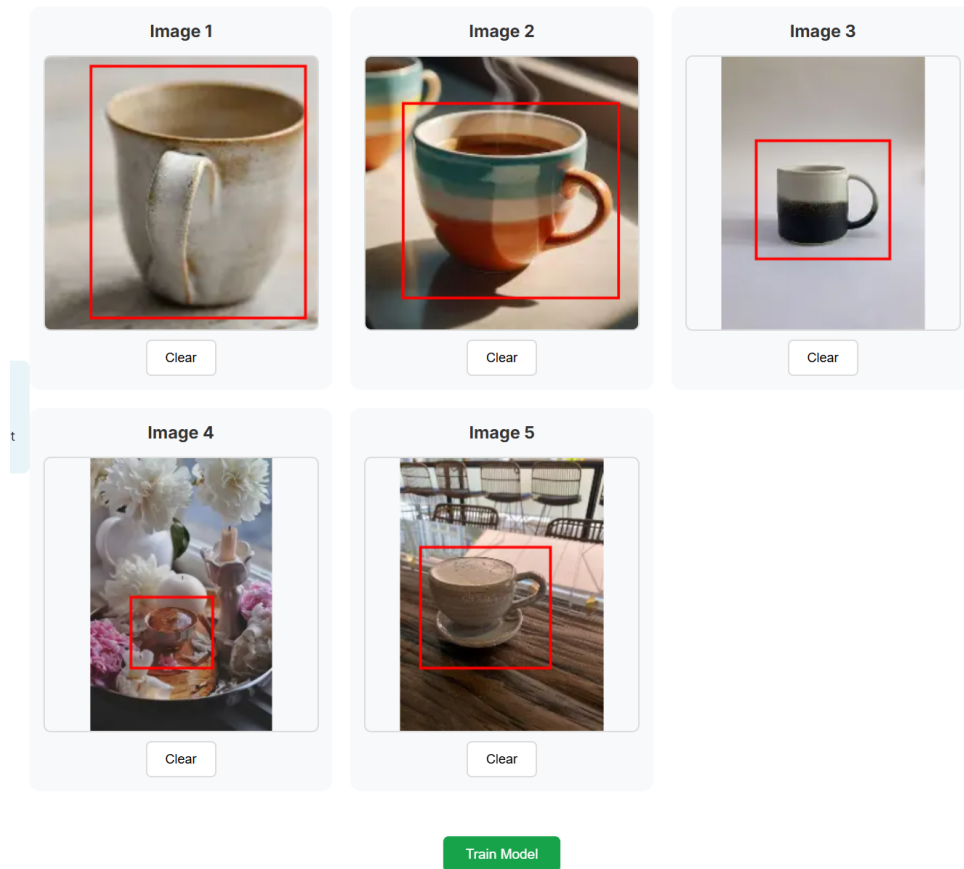


Figure 3: Screenshot of the annotation canvas interface showing bounding box drawing in progress. Users click and drag to create boxes.

Figure 3 shows the annotation canvas in action, demonstrating the bounding box drawing workflow that enables users to manually label objects of interest.

3.3.2 Auto-Annotation: Class-Filtered YOLOv8m Pipeline

The auto-annotation feature uses the configured YOLO model (YOLOv8m by default), pretrained on COCO, for conservative target-class proposals on scraped images:

1. **Preprocessing.** Ultralytics performs letterbox resizing, tensor conversion, and pixel scaling for inference.
2. **Inference.** Ultralytics inference uses a confidence threshold of 0.35.
3. **Post-processing.** Filter by class, convert $xyxy \rightarrow xywh$, map COCO class ID to user label.

For objects outside the 80 COCO categories, automatic annotation is disabled rather than assigning misleading boxes. The model then trains from the human-annotated images only.

3.4 One-Click Model Training

Description. With a curated and annotated dataset, users initiate training with a single click.

Technical Implementation.

- Annotation conversion from JSON to YOLO format.
- Dataset YAML generation with train/val paths.
- YOLOv8m training via Ultralytics API (up to 75 epochs, automatic batch size).
- A background worker thread performs training while the browser polls a JSON status endpoint every two seconds.

3.4.1 Training Configuration and Hyperparameters

Table 5: YOLOv8m Training Hyperparameters (Default Configuration)

Parameter	Default	Description
Model Variant	YOLOv8m	Medium accuracy-first backbone (25.9M parameters)
Image Resolution	640×640	Input image size
Epochs	75	Maximum training passes over dataset
Batch Size	Auto (16/8/4)	Adjusted dynamically based on available VRAM
Optimizer	Auto	Selected by the installed Ultralytics release
Box Loss Gain	7.5	Bounding box regression loss weight
Class Loss Gain	0.5	Classification loss weight
DFL Loss Gain	1.5	Distribution Focal Loss weight
Early-Stopping Patience	10	Stop when validation fitness ceases improving
Validation Split	20%	Deterministic held-out labeled images

Model Selection Rationale. YOLOv8m is the accuracy-first default:

- **Localization Quality.** On the project’s July 2026 COCO8 smoke validation, pretrained YOLOv8m achieved 74.0% mAP50–95 and 92.8% mAP50.
- **Transfer Learning.** COCO-pretrained weights provide stronger localization priors than the nano model for supported categories.
- **Configurable Cost.** `YOLO_MODEL=yolov8n.pt` restores the lightweight model when memory or latency is more important than accuracy.

The benchmark contains only four training and four validation images and is a pipeline smoke test, not evidence that arbitrary custom classes will reach the same score. A custom model’s accuracy remains dominated by dataset size, annotation quality, and distribution match.

3.4.2 Validation Method and July 2026 Smoke Benchmark

SearchVision moves a deterministic 20% of labeled image/label pairs into a distinct validation directory before training. Ultralytics reports precision, recall, mAP50, and mAP50–95 on this held-out subset. The following small COCO8 comparison was used to choose the default backbone; all values are validation metrics.

Table 6: Accuracy-oriented smoke benchmark on COCO8

Model	Precision	Recall	mAP50	mAP50–95
YOLOv8n fine-tuned	0.780	0.750	0.875	0.641
YOLOv8m pretrained validation	0.811	0.853	0.928	0.740

Because COCO8 is extremely small, a single missed detection can move these values substantially. The benchmark verifies the software path and motivates the default; it is not a statistically robust estimate of deployment performance.

To evaluate the value of automated dataset expansion separately from backbone choice, a second controlled benchmark used the COCO `person` class. The seed arm contained five training images, matching SearchVision’s minimum human-annotation workflow. The expansion arm contained the same five images plus 25 additional images, matching a successful automated scrape-and-annotation pass. Both arms used identical COCO-pretrained YOLOv8n weights, 20 epochs, a fixed seed of 42, and the same disjoint 20-image validation set (93 person instances). Metrics below are from the checkpoint with the highest validation mAP50–95 in each arm.

Table 7: Controlled single-class dataset-expansion benchmark

Training set	Images	Precision	Recall	mAP50	mAP50–95
Human seed only	5	0.011	0.688	0.258	0.203
Seed + automated expansion	30	0.672	0.452	0.511	0.293
Relative mAP change	6×	—	—	+98.2%	+44.2%

The 25-image expansion nearly doubled mAP50 and improved mAP50–95 by 44.2%, while also producing substantially better-calibrated precision. Recall alone was higher in the five-image arm because its best-mAP checkpoint emitted many low-quality detections; its 0.011 precision and lower mAP show that this is not a useful operating point. This benchmark supports the value of expanding beyond the five-image seed when additional images receive correct labels. It is deliberately an upper-bound test of the expansion stage: reviewed COCO labels stand in for successful automatic annotations, so the experiment does not measure search relevance, download yield, or pseudo-label errors on live web imagery. The complete benchmark can be reproduced with `python scripts/run_expansion_benchmark.py -epochs 20`.

3.4.3 Data Augmentation and Small-Dataset Handling

YOLOv8 applies data augmentation during training to improve generalization on small datasets. The default YOLOv8 augmentation pipeline includes:

1. **Mosaic Augmentation.** Four images stitched into one sample with random scaling and positioning. Increases training density and improves robustness to small objects.
2. **Geometric Transforms.**
 - Random translation: $\pm 10\%$ of image size
 - Random scaling according to the installed Ultralytics defaults
3. **Color Jitter.** Hue, saturation, and value shifts in HSV color space.
4. **Input Processing.** Ultralytics handles resizing, tensor conversion, and pixel scaling internally.

Note on Default Augmentation: The current implementation uses YOLOv8’s default augmentation settings. Some augmentation techniques (MixUp, Copy-Paste, rotation) are disabled by default in YOLOv8. These could be enabled in future versions to further improve small-dataset training.

Empirical Observations. In testing:

- Transfer learning is important for the small datasets targeted by the application.
- A real held-out split prevents direct train/validation leakage, although very small validation sets still produce high-variance metrics.

SearchVision prioritizes proof-of-concept and rapid prototyping over production-grade training. Users seeking higher accuracy should use larger datasets (100+ images) and enable additional augmentation techniques.

4. Technology Stack

Table 8: SearchVision Technology Stack

Category	Technology	Role
Web Framework	FastAPI	REST API backend, async request handling
ASGI Server	Uvicorn	Production-grade ASGI server
Frontend	HTML5, CSS3, JavaScript	User interface; Canvas API for annotation
Templating	Jinja2	Server-side HTML rendering
Object Detection	Ultralytics YOLOv8	Pre-trained and custom model training
Deep Learning	PyTorch	Neural network backend
Computer Vision	TorchVision	ResNet50 feature extraction
Feature Matching	Scikit-learn	Cosine distance computation
Image Search	Google Custom Search	Primary image discovery API
Image Search	Bing Images	Fallback image source (free, scrapes HTML)
HTTP Client	Requests	API communication, HTML fetching
Image Processing	Pillow (PIL)	Image loading, validation, preprocessing
Data Processing	NumPy	Array operations, numerical computing
Configuration	python-dotenv	Environment variable management

5. Related Work

The landscape of tools for computer vision dataset creation and model training has evolved significantly in recent years. This section situates SearchVision within the broader ecosystem of annotation platforms, AutoML solutions, and end-to-end pipelines.

5.1 Annotation Platforms

CVAT [CVAT, 2024] is an open-source annotation platform developed by Intel that supports bounding box, polygon, and polyline annotations for images and videos. CVAT excels at collaborative labeling workflows and provides sophisticated quality control mechanisms but does not integrate dataset collection or model training.

Label Studio [Label Studio, 2024] is a flexible open-source data labeling platform supporting over 20 data types and annotation modalities. Like CVAT, it focuses exclusively on the annotation phase and requires external tooling for dataset acquisition and model training.

Roboflow [Roboflow, 2024] is a commercial platform offering dataset management, augmentation, and model training with a web-based interface. While it provides more integration than open-source alternatives, it does not automate the initial dataset collection phase and operates as a SaaS platform with subscription costs.

SearchVision differentiates itself by providing a **complete end-to-end pipeline** that combines dataset collection (via search and scraping), diversity-aware image selection, dual-mode annotation, and training in a single self-hosted application. This contrasts with point solutions that require users to chain multiple tools together.

5.2 AutoML and Cloud-Based Training

Cloud-based services including Google Cloud AutoML Vision, Amazon Rekognition Custom Labels, and Microsoft Azure Custom Vision provide fully managed model training. These platforms accept user-provided images and return trained models with minimal configuration. However:

- **Cost.** Per-image or per-training-run fees can accumulate quickly.
- **Data Privacy.** User images and models are stored on cloud infrastructure.
- **Cold-Start Problem.** These services require users to provide a dataset; they do not assist with dataset collection.

SearchVision provides a self-hosted alternative that gives users full control over data, training, and model ownership while avoiding per-use costs.

5.3 Dataset Search and Analysis Tools

FiftyOne [FiftyOne, 2024] is an open-source visualization and evaluation tool that helps users analyze datasets, explore model predictions, and identify edge cases. While it excels at dataset analysis post-training, it does not address dataset creation or collection.

Recent work on synthetic data generation and dataset distillation has explored meth-

ods for creating compact training sets that approximate larger datasets. However, these approaches require existing large datasets as source material and do not address the cold-start problem of collecting the initial dataset from scratch. SearchVision’s contribution lies specifically in automating this cold-start phase through web search and scraping.

5.4 Web Scraping and Image Collection

Numerous tools and libraries enable web scraping (BeautifulSoup, Scrapy, Selenium) and image collection (bing-image-downloader, google-images-download). However, these tools focus on the data collection phase exclusively and do not integrate annotation or training. SearchVision differentiates by embedding web scraping as one component within a larger, cohesive pipeline.

6. Use Cases

The following scenarios are illustrative applications, not reported deployments or measured customer outcomes. Their feasibility and timing depend on the available images, class, and hardware.

6.1 Rapid Prototyping

Context. A pre-seed startup is exploring computer vision solutions for retail analytics. Before committing engineering resources to a full machine learning project, the founding team needs to validate whether state-of-the-art object detection models can achieve acceptable performance on their specific use case: detecting out-of-stock products on retail shelves.

Scenario. The team has identified the core technical hypothesis: a YOLOv8 model fine-tuned on retail shelf imagery can achieve $>80\%$ mAP on product detection. Testing this hypothesis requires a labeled dataset, which the team does not currently possess. Traditional dataset creation would require weeks of data collection and labeling.

Workflow.

1. Enter the search query “retail shelf products” to retrieve initial image candidates.
2. Review the diversity-sorted results and select nine images representing various shelf configurations, lighting conditions, and product arrangements.
3. Draw human ground-truth boxes on the selected images; automatic annotation is

subsequently applied only to eligible scraped images.

4. Initiate model training with default hyperparameters.

Potential Outcome. The team obtains a baseline model and held-out metrics that can inform whether collecting a larger, domain-owned dataset is worthwhile. The tool alone does not establish production feasibility.

6.2 Startup Bootstrapping

Context. A manufacturing company has identified surface defect detection as a priority for their quality control pipeline. Their in-house team consists of mechanical engineers and production specialists, none of whom have machine learning experience. Purchasing commercial AutoML solutions would exceed their project budget.

Scenario. The quality control team needs to detect scratches, dents, and discoloration on manufactured components. The products vary significantly in material (painted metal, plastic, raw metal), requiring a diverse training set that captures this variation.

Workflow.

1. Choose one defect class and one focused query for a single-class training run.
2. Select diverse results for that class; repeat the workflow separately for other classes because the current application does not aggregate queries or train multiple classes.
3. Draw ground-truth boxes on selected images. For these non-COCO defect classes, additional scraped images require human labels outside the current automated flow.
4. Train the model and export weights for deployment to edge inference devices on the production floor.

Potential Outcome. The workflow can establish a prototype, but defect categories are generally outside COCO and therefore require substantially more human annotation and independent testing before quality-control use.

6.3 Educational Purposes

Context. A university course on computer vision attracts students from diverse backgrounds, including computer science, electrical engineering, and data science. The instructor seeks to provide hands-on experience with the complete ML pipeline without requiring students to master multiple frameworks and tools.

Scenario. Students complete a semester-long project applying object detection to a domain of their choosing. Projects have included wildlife detection in camera trap images, medical device localization in X-ray photographs, and traffic sign recognition.

Workflow.

1. Students define object categories relevant to their project (e.g., specific bird species, medical instrument types, traffic sign variants).
2. Experiment with search queries to understand how query formulation affects dataset quality and relevance.
3. Compare manual annotation coverage with eligible automatic labels by inspecting generated label files outside the current UI.
4. Train models with the documented defaults and analyze mAP, precision, and recall; advanced hyperparameter experiments require code or environment changes.
5. Write technical reports documenting the end-to-end pipeline.

Potential Outcome. Students can gain practical experience with dataset curation, annotation quality, leakage-free evaluation, and model training through one interface.

6.4 Research Iteration

Context. A graduate researcher is investigating few-shot learning techniques for novel object categories. The core challenge is rapidly generating training data for objects not well-represented in existing benchmarks, such as specific botanical specimens or architectural features.

Scenario. The research involves evaluating whether pre-trained vision-language models can accelerate dataset creation for rare object categories. The researcher needs to quickly generate training and validation sets for multiple categories to conduct comparative experiments.

Workflow.

1. Query specific categories (e.g., “monarch butterfly,” “gothic arch window”).
2. Apply diversity selection to ensure the initial set captures viewpoint variation.
3. Manually annotate novel categories, since conservative automatic annotation is disabled when the query does not map to a COCO class.
4. Train a baseline YOLOv8 model using the built-in validation split, then evaluate independently on a researcher-maintained test set.

5. Repeat separate single-class runs with refined queries and annotations; detailed confusion-matrix analysis uses the artifacts emitted by Ultralytics outside the UI.

Potential Outcome. The researcher can collect and manually seed a novel-class dataset more quickly, while retaining responsibility for sample size, label review, and an independent test protocol.

7. Limitations and Known Issues

SearchVision acknowledges the following limitations and known issues that users should be aware of:

1. **Limited Image Quality Controls.**

- Pillow decoding rejects non-images and corrupt payloads, and accepted images are normalized to RGB JPEG.
- There is no minimum-resolution filter, byte-size ceiling, or duplicate-image detector beyond exact URL deduplication.

2. **Small Validation Sets.**

- A distinct deterministic 80/20 split removes direct leakage.
- With only five or twenty images, the validation subset is too small for a stable generalization estimate; a separate external test set is still recommended.

3. **Google Custom Search API Dependency.**

- **Limitation:** Searches are limited by Google Custom Search API quotas; high-volume usage requires paid plans. Some users may have API access disabled.
- **Mitigation:** Automatic fallback to Bing Images scraping (no API key required). Bing provides robust, consistent access.

4. **One-Pass Selection.**

- The image selection algorithm computes scores once rather than greedily selecting images to maximize diversity.
- Less robust diversity guarantee compared to iterative Max-Min approach.

- Impact: Selected images may have lower visual diversity than theoretical optimum.

5. Limited Data Augmentation.

- Current implementation uses YOLOv8's default augmentation (mosaic, translation, color jitter).
- Advanced techniques (MixUp, Copy-Paste, rotation) are disabled by default.
- Impact: Less effective training on very small datasets.

6. Search Relevance Variability.

- Result quality depends on query formulation; niche objects may yield irrelevant imagery.
- Mitigation: Users can click “Search Again” for pagination through different result sets.

7. Scraping Rate Limits and Timeouts.

- Web scraping inherently depends on target website availability.
- Graceful fallback: If scraping fails, training continues with manual annotations only.

8. Auto-Annotation Limitations.

- The pre-trained detector recognizes only COCO classes plus a small alias map.
- Unknown classes are deliberately not auto-annotated; target misses are excluded instead of becoming false-negative labels.
- Auto-generated scraped labels are not currently exposed for interactive correction.

9. Training Compute Constraints.

- Local-only training; CPU-only machines are substantially slower than GPU-equipped hardware.
- Default YOLOv8m improves accuracy but is substantially slower and larger than nano.
- Resource-constrained users can set `YOLO_MODEL=yolov8n.pt`.

10. **Single-Class Optimization.**

- Current implementation is optimized for single-object detection; multi-class requires manual configuration or multiple training runs.

11. **No Model Versioning.**

- Trained weights are saved but not systematically versioned or tracked.
- Manual file organization enables simple versioning by users.

12. **Single-User Process State.**

- Training status and the active dataset are process-global; concurrent users or multiple workers are not supported safely.
- Provides data privacy and full control over training.

13. **Limited Export Formats.**

- Output is PyTorch format (.pt); ONNX, TFLite, and Core ML require manual conversion via Ultralytics export API.

8. Conclusion

SearchVision represents a functional system for rapid prototyping of object detection models by integrating search, image selection, annotation, and training into a single workflow.

System Capabilities:

1. **End-to-End Automation.** Complete integration from query to deployable model, eliminating context-switching between tools.
2. **Hybrid Image Selection.** Combines search ranking with visual feature-based dissimilarity to select diverse results.
3. **Graceful API Fallback.** Automatic fallback from Google Custom Search to Bing Images ensures accessibility.
4. **Conservative Annotation.** Preserves human ground truth and adds automatic labels only for recognized target classes on scraped images.

5. **Leakage-Free Evaluation.** Uses distinct train and validation directories and reports mAP50 and mAP50–95.

Appropriate Use Cases:

- Rapid prototyping for proof-of-concept validation
- Educational purposes (learning ML pipelines)
- Quick feasibility assessment before larger ML projects
- Small, single-class experiments where users understand metric uncertainty

Known Limitations Affecting Reliability:

The system has two important limitations that users must understand:

1. **Limited Statistical Power:** The split is leakage-free, but a tiny validation subset cannot provide a precise estimate of deployment performance.
2. **Limited Quality Screening:** Decode validation is implemented, but minimum resolution, near-duplicate detection, and human review of automatic labels are not.

For production deployments, users should:

- Maintain an independent, representative test set
- Add minimum-resolution and near-duplicate quality checks
- Consider larger datasets (100+ images) for improved reliability
- Enable additional augmentation techniques for better small-dataset learning
- Manually evaluate model performance on diverse test data

Summary: SearchVision successfully reduces tooling overhead for dataset collection and model training but remains best suited to exploratory, educational, and proof-of-concept work. The current validation and download safeguards make its metrics more honest than the older version, but production use still requires more data, external testing, review of automatic labels, and multi-user state isolation.

References

- [CVAT, 2024] CVAT (2024). *Computer Vision Annotation Tool*. Intel. <https://github.com/openopencv/cvat>
- [FiftyOne, 2024] FiftyOne (2024). *The Open-Source Tool for Building High-Quality Datasets and Computer Vision Models*. Voxel51. <https://github.com/voxel51/fiftyone>
- [Label Studio, 2024] Label Studio (2024). *Label Studio: The Open Source Data Labeling Platform*. HumanSignal. <https://labelstud.io/>
- [Redmon and Farhadi, 2018] Redmon, J. and Farhadi, A. (2018). YOLOv3: An Incremental Improvement. *arXiv preprint arXiv:1804.02767*.
- [Ren et al., 2015] Ren, S., He, K., Girshick, R., and Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *Advances in Neural Information Processing Systems*, 28, 91–99.
- [Roboflow, 2024] Roboflow (2024). *Roboflow: The Computer Vision Platform*. <https://roboflow.com/>
- [Ultralytics, 2024] Ultralytics (2024). *YOLOv8: State-of-the-Art Object Detection Models*. <https://github.com/ultralytics/ultralytics>
- [He et al., 2016] He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep Residual Learning for Image Recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.
- [Miller, 1995] Miller, G. A. (1995). WordNet: A Lexical Database for English. *Communications of the ACM*, 38(11), 39–41.

Acknowledgements

SearchVision is an open-source project licensed under AGPL-3.0. The author acknowledges the open-source communities behind PyTorch, YOLOv8, and the various libraries that make this work possible.